This is a retrospective working log for the technical assessment. I'm recording this after I finished building. To be honest, I downloaded Wispr Flow before the assessment started, but didn't actually use it during the build. I went into focus mode, and once the brief dropped, I did forget to use Wispr Flow. I'm doing this after, and the text is real. I'd say the timestamps are approximate.

When the brief dropped yesterday, the first thing I did was read it twice.

- Soft drinks category
- 130 transcripts
- 35 products
- 9 brands
- 4 brands with transcripts
- 5 without

The line that decided everything for me was the one that said, "If your app suggests a tag, attribute, theme, or signal, it should show the evidence behind it." That was the technical centerpiece, and I knew that that's where I wanted to spend most of my time thinking.

The other thing that stood out for me was the framing. It was mentioned that it's not just displaying files. The job was to make a useful intelligence tool, something that connects unscripted, unstructured transcript voice with the structured brand and product data. I decided the dashboard had to have a join in it, not just two separate sections, but some place where the two data types actually meet on the page.

I also realized the transcript coverage was partial by design. For the 5 brands that didn't have transcripts, that wasn't a missing data problem. That was the data set telling me to think about the category context too. The dashboard had to handle both.

Going into the build, I committed to a few things:

- Quickly react for the frontend. I've used React before, and Bit is fast.
- A fast API for the backend, and Python is my strongest language. Fast API is easy.
- No database, with only 130 transcripts and 35 products. Loading everything into memory at startup was simpler and faster than wiring SQLite for no real benefit.
- Hosting on Vercel for the frontend and Render free tier for the backend, both connected to my GitHub directly.

The biggest stack decision was about the AI extraction. I had two options:

- Call the LLM every time the user hits an endpoint.
- Extract everything once offline and save it as a JSON file the backend then reads from.

I went with offline for four reasons:

- It's cheaper. Extraction costs were only about $0.40 in total.

- It's faster. There is no latency on page load.
- It's reproducible. The JSON is in the repo.
- It's auditable. Anyone can see what was extracted.

The extracted extraction script I ran late on Wednesday evening. It actually finished about 3 a.m. because I was at a concert in between. I wrote a Python script that takes each transcript, sends it to Claude Sonnet with a very structured prompt, and gets back a JSON object with:

- Themes
- Praise
- Complaints
- Usage occasions and competitor mentions
- Purchase drivers
- A headline quote

Each one has to come with the exact phrase from the transcript that supports it. I made the prompt explicit about the rules. If a category has nothing in the transcript, it has to return an empty array and not invent it. Evidence has to be the exact phrase, not a paraphrase. I kept the tag short. I tested it with a few transcripts before running the whole batch. The first one was the Double Dutch Lemonade review where the reviewer compares it to Shweppes (I think that's how you pronounce it). The extraction picked up the themes:

- Head-to-head comparison
- Natural taste preference

The praise was:

- No artificial aftertaste
- More gentle

  The complaints were:

  - Slightly bitter
  - Not as strong

    The competitor mention as well, all with exact quotes, so I trusted it enough to run for the full 133. The script saved progress every 10 transcripts so a crash wouldn't lose work, and it took about 10 minutes in total, which, like I mentioned before, cost 40 cents. The output ended up as a 300 KB JSON file in the data folder.

What did I build? I built three pages, each answering a different question.

    - The brand overview is the landing page. It has a breakthrough score bar chart for all nine brands, with the reviewed brands in blue and the competitor ones in grey. The visual difference immediately tells the user these are the ones we have voice data for. These are just

context. Below that are two grids: reviewed brands with transcript counts, competitor brands without. The first thing the user notices is the highest scoring brands actually don't have transcripts. That's the commercial story the dashboard exists to address.

The brand detail page is where the real work happens. For a brand with transcripts, the centerpiece is the positioning versus consumer voice panel side by side. The brand's stated positioning fromfrom the product data against what reviewers actually against what reviews actually said, based on extracted signals. Then below that there's:

- Praise with evidence quotes
- Complaints with evidence quotes
- Competitor mentions and the products

For brand without transcripts, I just show the positioning context, so I deliberately don't fake it.

The transcript browser is the third page. It's a filter list:

- Search by product or brand or headline quote
- Filter by sentiment

You can click on a transcript and you can get the full text with evidence phrases highlighted in yellow. You can see literally which sentence backs which tag, and that's the audit trail.

The things that went wrong, sort of in order, were:

1. The Jason Nunn bug. I'd loaded the products as CSV into Pandas and then tried to send the data across as JSON and caught 500 errors. I looked at the trace and Pandas was returning None for empty cells, which Jason dumps doesn't allow. I tried to fix it with `df.where(pd.notnull(df), None)`, but that doesn't always reach every cell when column tags have, of course, mixed types. I switched it over, and that did solve it. The 5-minute fix once I did read the trace.
2. Deploying render. The build succeeded when I opened the URL, but it said not found. The URL I had been using was the RGC assessment backend on render.com, and because that's what I named the service, Render had actually deployed it as RGC assessment on render.com and uses the GitHub repo name, not the service name, in the YAML config. I just used the right URL.
3. When I was making some of my submission notes, I had an error where the LaTeX, which was 3GB, refused to install. I worked around it by converting it to HTML, opening the browser, and then saving it as a PDF that way. That only took about less than 2 min.
4. Some of the smaller stuff included: I got confused between my terminal windows at times because this was the first time I was running multiple terminal windows at the same time. It took a minute to sort of realize what was happening, so the new uv core uv icon server was running fine in one, and then I just needed to run another one for my Git commands.

5. Looking at how I used AI, I use Claude pretty heavily throughout to help sort of help my workflow and also help with maximizing my time. Everything was with my verification, so verification was fully mine. Signal extraction was done by Claude, the engine doing that work. I wrote the prompt, a very precise prompt, ran batch and spot check on any outputs for pair programming. I had a Claude conversation open alongside the build, so I used it for the fast API endpoint design, the React components, folding the CSS, decisions, and the debugging, particularly the JSON nan bug and the render URL issue.
6. For submission notes, I drafted everything myself. I had this neat and upped by Claude, and then it was edited for accuracy, voice, and any specific insights that I claimed. I checked these under my actual data before committing them under my name.
7. I would basically say Claude was my default for typing out boilerplate and being sort of a rubber duck, and verification was fully stuck with me.
8. What would I do if I had more time? The top of my list would be an automated agreement scoring between the brand's positioning and what reviewers said. Right now I show both sides, and the user reads it themselves. With more time, I would compute a match score so the dashboard could say the brand's positioning is, say, 68% consistent with the reviewer voice.
9. Aspect-level sentiment: instead of one sentiment per transcript extract, extract the sentiment per dimension (taste, price, packet, packaging, occasion fit). This gives a much richer brand-level view and a "what to investigate next" insight card, kind of like an LLM-generated short narrative per brand that synthesises the gap into actionable observations.
10. Reviewer segments: I didn't touch the user's JSON at all. The shopper architects and behavioural tags would let me answer questions like which segments love skip? That's a clear gap I'd want to close.
11. Honestly test for extraction prompt: right now I trust that the JSON parsing succeeds, but with more time I'd add retry logic and validation.
12. To come to an end, my two final thoughts are:
    1. The thing that I'm most pleased with is the evidence highlighting on the transcript page. The brief specifically asked for evidence behind extracted signals, and seeing the yellow highlight on the supporting phrase right there in the original text makes the whole AI layer auditable rather than a black box. That's the bit I'd point to first if I was defending this.
    2. The thing I'd genuinely do differently, and that does include using Wispr Flow and having a discipline of talking out loud whilst I'm working, catches sort of my own confused thinking that I might have had earlier. I did lose an hour or two on useful self-correction by not having that running, so consider this a real example of one of the things to improve next time.